

1 Introduction to Neural Operators

Neural operators are a class of machine learning models designed to learn mappings between infinite-dimensional function spaces. In contrast to conventional neural networks, which generally learn mappings between finite-dimensional vectors, neural operators aim to approximate an operator

$$\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U}, \quad (1)$$

where $\mathcal{A}$ and $\mathcal{U}$ are spaces of functions. Given an input function $a(x)$, a neural operator learns to predict an output function $u(x)$,

$$u = \mathcal{G}(a). \quad (2)$$

This formulation is particularly relevant for scientific computing because many physical and chemical systems are naturally described by differential equations whose solutions are themselves functions.

1.1 Neural Operators for Scientific Problems

A large class of problems in physics and chemistry can be formulated through parameterized differential equations. Consider a general partial differential equation

$$\mathcal{L}(u; a) = f, \quad (3)$$

where $a(x)$ represents spatially varying coefficients or material properties, $f(x)$ is a forcing term, and $u(x)$ is the resulting physical field. Solving this equation for every new realization of $a(x)$ can be computationally expensive, particularly when the underlying simulation requires fine spatial or temporal discretization.

A neural operator attempts to learn the solution operator directly:

$$\mathcal{G} : a(x) \mapsto u(x). \quad (4)$$

After training, the learned operator can evaluate the solution for new input functions without explicitly performing the complete numerical solution procedure each time. This makes neural operators particularly attractive for problems requiring repeated forward simulations, parameter estimation, optimization, uncertainty quantification, and real-time prediction.

1.2 Relevance to Physics

Physics provides a natural setting for operator learning because physical laws are frequently expressed as differential equations. Examples include the Navier–Stokes equations for fluid dynamics, Maxwell’s equations for electromagnetism, the Schrödinger equation for quantum systems, the heat equation for thermal transport, and elasticity equations for solid mechanics.

For example, consider the parameterized diffusion equation

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x), \quad (5)$$

with appropriate boundary conditions. The coefficient field $a(x)$ may represent a heterogeneous material property, while $u(x)$ represents the resulting physical field. Rather than learning a finite-dimensional mapping for one particular discretization, a neural operator seeks to approximate the underlying mapping

$$a(\cdot) \mapsto u(\cdot). \quad (6)$$

This distinction is important for scientific machine learning because the objective is not merely to predict individual numerical values. The objective is to learn the transformation between entire physical fields.

1.3 Relevance to Chemistry

Neural operators are also relevant to computational chemistry and molecular science, particularly when the quantities of interest are spatially distributed fields or solutions of parameterized differential equations. Examples include electronic structure, molecular potentials, reaction-diffusion systems, transport phenomena, quantum dynamics, and multiscale simulations.

Many computational chemistry problems require repeatedly solving expensive equations for different molecular configurations, external potentials, material parameters, or boundary conditions. An operator-learning model can potentially learn the mapping between these input fields or parameters and the corresponding physical solution. This creates the possibility of constructing fast surrogate models for expensive computational procedures while retaining a representation that is naturally compatible with continuous physical fields.

1.4 How Neural Operators Differ from Standard Deep Learning

The fundamental distinction between neural operators and conventional deep learning lies in the type of mapping being learned. A conventional neural network typically approximates a finite-dimensional function

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m. \quad (7)$$

For example, a standard neural network may take a vector of measurements

$$\mathbf{x} = (x_1, x_2, \dots, x_n) \quad (8)$$

and produce a finite-dimensional prediction $\mathbf{y} \in \mathbb{R}^m$.

A neural operator instead aims to approximate a mapping between functions:

$$\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U}. \quad (9)$$

Thus, its input and output can themselves be spatial or temporal fields.

This difference also affects discretization. A conventional neural network is generally tied to the dimensionality and representation of its input. Changing the spatial resolution of a discretized field can therefore require modifying the

network architecture or retraining the model. Neural operators are designed with a different objective: to approximate the underlying continuous operator rather than merely memorizing a mapping between particular finite-dimensional discretizations.

1.5 Operator Learning Perspective

Suppose a physical field $a(x)$ is discretized at N spatial locations. A conventional neural network may interpret the resulting vector

$$\mathbf{a} \in \mathbb{R}^N \quad (10)$$

as a collection of N numerical features. A neural operator instead attempts to preserve the interpretation of $\mathbf{a}$ as a discretization of an underlying function $a(x)$.

Conceptually, the distinction can be summarized as

$$\boxed{\text{Standard Deep Learning: } \mathbf{x} \mapsto \mathbf{y}} \quad (11)$$

whereas

$$\boxed{\text{Neural Operator: } a(\cdot) \mapsto u(\cdot)}. \quad (12)$$

This operator-level perspective allows neural operators to be used as surrogate models for families of related physical problems rather than as predictors for a single fixed discretized problem.

1.6 Why Neural Operators Matter

The central motivation behind neural operators is therefore not simply to replace conventional neural networks with a different architecture. Rather, they provide a framework for learning the solution operators underlying families of scientific problems.

Once a neural operator has been trained on sufficiently representative examples, it can potentially provide rapid predictions for previously unseen input functions. This can substantially reduce the computational cost of repeated simulations and enable applications such as inverse problems, optimization, control, uncertainty quantification, and parameter exploration.

Prominent neural-operator architectures include the Deep Operator Network (DeepONet), Fourier Neural Operator (FNO), Graph Neural Operator (GNO), and Physics-Informed Neural Operator (PINO). These approaches differ in how they represent and approximate the operator, but they share the central objective of learning mappings between function spaces.

Consequently, neural operators occupy an important position at the intersection of deep learning, numerical analysis, and scientific computing. Their ability to learn operators rather than isolated finite-dimensional mappings makes them particularly promising for physics and chemistry, where the underlying objects of interest are frequently continuous fields governed by mathematical laws.